

A1 Project Guide

What This Project Is Really About

Large Language Models (LLMs) are fluent and convincing, but they often generate statements that are:

- factually incorrect,
- logically inconsistent,
- incompatible with domain rules, or
- internally contradictory.

These errors are commonly referred to as hallucinations.

This project is not about training or fine-tuning an LLM to reduce hallucinations. Instead, it focuses on a different question:

Can we detect hallucinations by checking LLM outputs against explicit symbolic knowledge?

Rather than trying to “fix” the model itself, the goal is to design a knowledge-grounded verification layer that evaluates its outputs using structured rules and constraints.

Core Research Question

The central research question is:

How can symbolic domain knowledge be used to detect logically or factually inconsistent LLM outputs?

More concretely:

- What kinds of hallucinations can be detected through constraint checking?
- Which hallucinations cannot be detected using symbolic verification?
- Does adding a reasoning layer significantly reduce undetected errors compared to the LLM alone?

What This Project Is NOT About

To keep the scope manageable, this project does not involve:

- Fine-tuning or retraining LLMs
- Working on open-domain factual correctness
- Building a full real-world knowledge graph
- Attempting to “solve hallucinations” in general

The project must focus on:

- A restricted domain
- Clearly formalizable rules
- Explicit constraint violation detection

Suggested Technical Approach

Step 1 — Select a Restricted Domain

Examples:

- Finance (loan eligibility rules, budget constraints)
- Policy (eligibility conditions, rule-based approvals)
- Games (legal moves, scoring rules)

The domain must:

- Have well-defined rules
- Allow symbolic formalization
- Avoid ambiguity

Keep it small and controlled.

Step 2 — Formalize Domain Knowledge

Represent domain knowledge as:

- Logical rules
- Constraints
- Structured facts
- A simple rule-based system

Examples:

- “A player cannot move to an occupied square.”
- “An applicant must be over 18.”
- “Expenses cannot exceed budget.”

Step 3 — Generate LLM Outputs

Design prompts that:

- Ask for reasoning within the selected domain
- Encourage multi-step reasoning
- Produce structured outputs if possible

The output format should make verification easier.

Step 4 — Implement the Verification Layer

Build a symbolic reasoning component that:

- Parses LLM outputs
- Checks consistency against domain constraints
- Flags violations

This may involve:

- Rule-based checking
- Logical evaluation
- Graph consistency checking

The goal is clarity, not complexity.

Step 5 — Categorize Hallucinations

Define clear categories, such as:

- Rule violations
- Logical inconsistencies
- Missing required conditions
- Fabricated domain facts

A taxonomy is important for evaluation.

Step 6 — Evaluation

Compare:

- LLM alone
- LLM + symbolic verification

Measure:

- Violation detection rate
- Types of hallucinations detected
- False positives

The evaluation must be systematic and controlled.

What a Good Thesis Should Demonstrate

A strong thesis will include:

- A clearly defined restricted domain
- A clean symbolic representation
- A working verification pipeline
- A structured taxonomy of hallucination types
- A careful empirical analysis of detection effectiveness

Clarity and rigor are more important than complexity.

Common Pitfalls to Avoid

- Choosing a domain that is too broad
- Overcomplicating the logical formalization
- Attempting open-domain factual verification
- Building an overly complex parsing layer

Keep the system small, structured, and controlled.



Starter Readings

Survey of Hallucination in Natural Language Generation

<https://arxiv.org/abs/2202.03629>

Provides taxonomy and classification of hallucinations.

A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions

<https://arxiv.org/abs/2311.05232>

Modern overview of hallucination causes, detection, and open research problems.

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (RAG)

<https://arxiv.org/abs/2005.11401>

Introduces knowledge augmentation architecture. Your project replaces retrieval with symbolic constraint checking.

SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models

<https://arxiv.org/abs/2303.08896>

Baseline hallucination detection approach without symbolic knowledge.

Adaptable Logical Control for Large Language Models

<https://arxiv.org/abs/2406.13892>

Modern approach to logical constraint enforcement in LLM outputs.